

Guía autónoma de taller: ESP32 + MIT App Inventor

Bluetooth, HTTP GET, HTTP POST/JSON y adquisición analógica en tiempo real

Propósito. Esta guía está diseñada para que cada equipo avance de manera autónoma. En cada ejercicio se valida primero el hardware, luego el firmware del ESP32, después la comunicación y finalmente la app. No continúe a la siguiente etapa si la anterior no funciona.

Regla de trabajo: Diagnostique por capas: 1) hardware, 2) código/ESP32, 3) comunicación, 4) MIT App Inventor. Cambiar varias cosas a la vez dificulta identificar el error.

Antes de empezar

Elemento	Requisito / recomendación
Arduino IDE	Placa seleccionada: ESP32 Dev Module; puerto COM correcto.
Monitor Serial	Usar 115200 baud cuando el código incluya <code>Serial.begin(115200)</code> .
MIT App Inventor	Crear un proyecto diferente por ejercicio. En Designer se agregan componentes; en Blocks se programa la lógica.
Wi-Fi	ESP32 y celular deben estar en la misma red local en los ejercicios 2, 3 y 4.
Bluetooth	El Ejercicio 1 usa Bluetooth clásico; emparejar el ESP32 desde Android antes de probar la app si fuera necesario.
Librerías	ArduinoJson y ESP32Servo se instalan desde Library Manager para el Ejercicio 3. WiFiManager se usa en la adaptación final del ejercicio asíncrono.

Materiales generales

Cantidad por equipo	Material
1	ESP32 DevKit V1 / ESP32-WROOM-32
1	Protoboard
Varios	Cables Dupont
2	LED de 5 mm
2–3	Resistencias 220–330 Ω
1	Servo (SG90 o equivalente para la práctica)
1	Potenciómetro
1	Flex sensor 2.2" (para el proyecto asíncrono)
1	Resistencia 47 k Ω ± 5 %
1	Cable USB de datos
1	Smartphone Android
1	Laptop con Arduino IDE y acceso a MIT App Inventor

Ejercicio 1 — Encender y apagar un LED por Bluetooth clásico

Objetivo. Controlar un LED conectado al ESP32 desde una app Android usando Bluetooth clásico. Este ejercicio introduce ListPicker, BluetoothClient, envío de texto y retroalimentación de conexión.

1.1 Materiales y conexión

Componente	Conexión
LED	Ánodo hacia GPIO 2 mediante resistencia de 220–330 Ω ; cátodo a GND.
ESP32	Alimentación por USB.
Celular	Bluetooth activado.

1.2 Firmware del ESP32

1. Abra **Arduino IDE**. Seleccione ESP32 Dev Module y el puerto COM de la placa.
2. Pegue el código del **Ejercicio 1**. El código se entrega en el documento separado de códigos.
3. Compile y cargue el sketch. No continúe si aparece un error.
4. Abra el **Monitor Serial a 115200**. Debe observar el mensaje de inicio de Bluetooth en la versión de taller.
5. Empareje el celular con **"ESP32_LED"**. Si Android solicita permiso o emparejamiento, acéptelo.

1.3 MIT App Inventor — Designer

6. Cree un proyecto nuevo, por ejemplo: Ejercicio1_Bluetooth_LED.
7. En User Interface arrastre un ListPicker. Renómbrelo btnConectar. Cambie Text a "CONECTAR BLUETOOTH".
8. Arrastre un Label. Renómbrelo lblEstado. Coloque Text = "OFFLINE".
9. Arrastre dos Button. Renómbrelos btnON y btnOFF. Text = "ON" y "OFF".
10. En Connectivity arrastre BluetoothClient. Aparecerá como componente no visible: BluetoothClient1.
11. Opcional: use HorizontalArrangement para ordenar los botones.

1.4 MIT App Inventor — Blocks

12. Desde btnConectar arrastre when btnConectar.BeforePicking.
13. Dentro coloque set btnConectar.Elements to BluetoothClient1.AddressesAndNames. Esto llena la lista con dispositivos Bluetooth emparejados.
14. Desde btnConectar arrastre when btnConectar.AfterPicking.
15. Llame BluetoothClient1.Connect con address = btnConectar.Selection.
16. Recomendado: use un bloque if con el resultado de Connect. En then: lblEstado.Text = "CONNECTED". En else: lblEstado.Text = "ERROR".
17. En when btnON.Click llame BluetoothClient1.SendText con text = "1".
18. En when btnOFF.Click llame BluetoothClient1.SendText con text = "0".

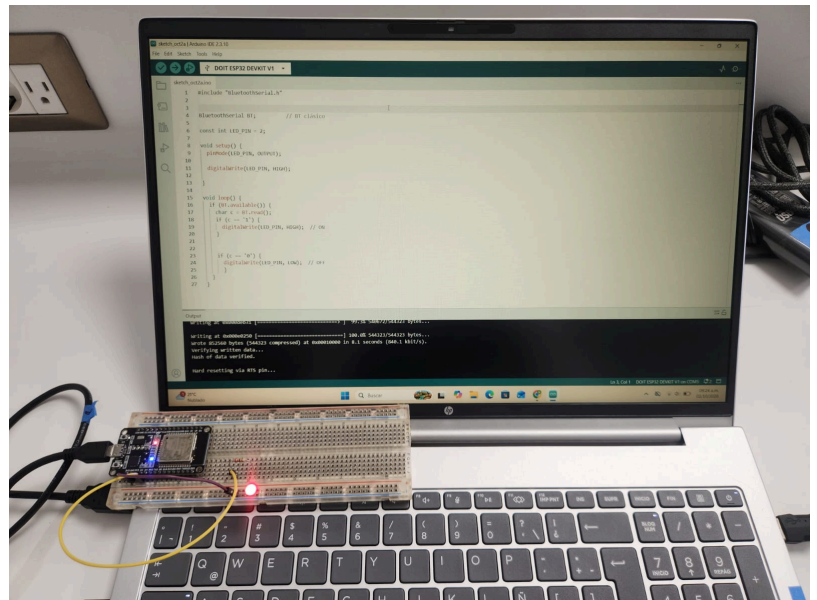
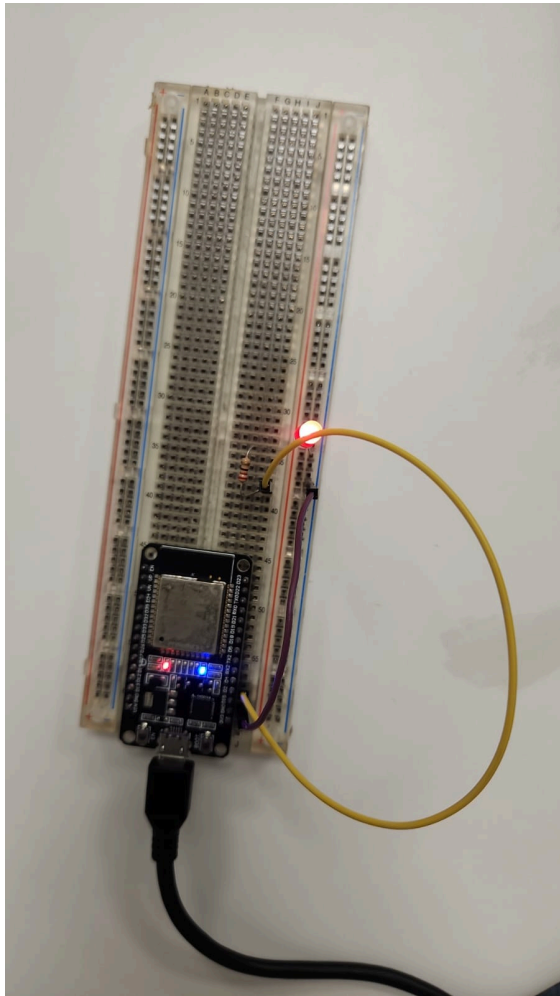
Qué hace cada bloque: BeforePicking prepara la lista antes de abrirla; AddressesAndNames obtiene los dispositivos emparejados; AfterPicking se ejecuta después de elegir uno; Connect intenta la conexión; SendText envía un carácter al ESP32. El Label de estado no enciende el LED: solo informa al usuario.

1.5 Prueba y descarte de errores

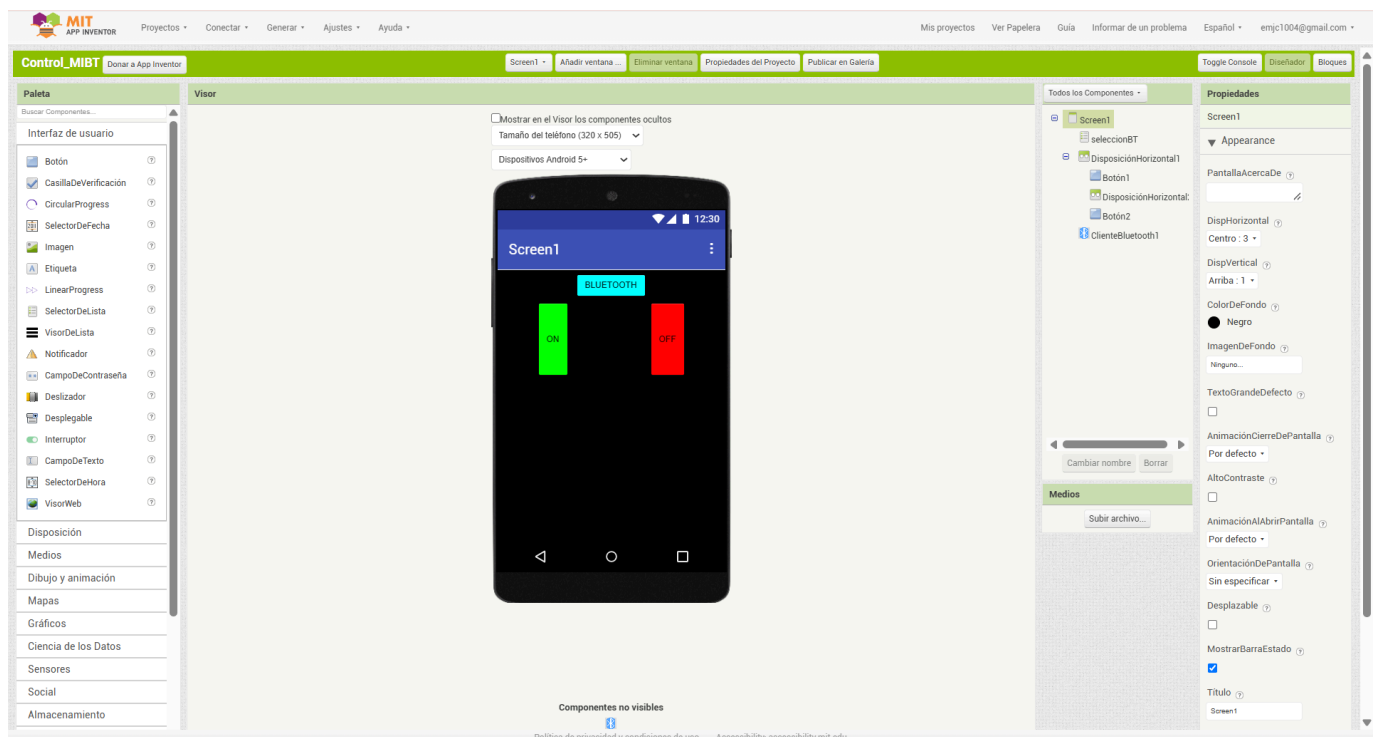
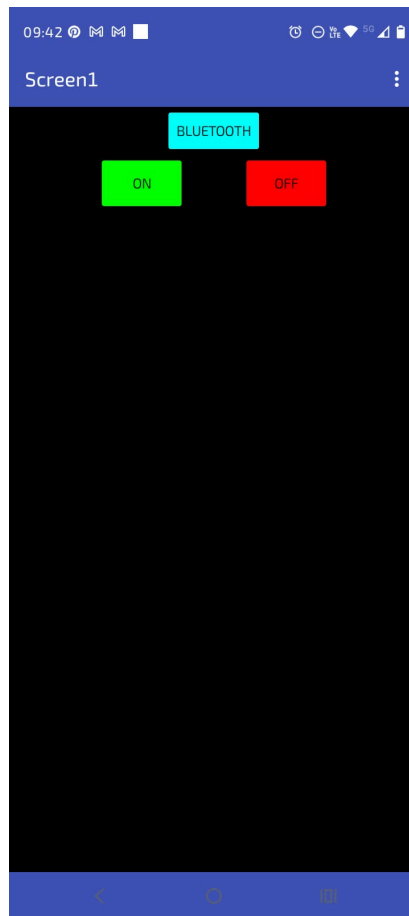
Orden	Prueba	Si falla, revisar
1	Prueba el LED con un sketch simple.	Polaridad, resistencia, GPIO 2, GND.
2	El celular encuentra "ESP32_LED".	Bluetooth del ESP32, Bluetooth del celular, emparejamiento.
3	ListPicker muestra dispositivos.	BeforePicking y AddressesAndNames; permisos de Bluetooth.
4	lblEstado cambia a CONNECTED.	Selection y BluetoothClient1.Connect.
5	ON/OFF cambia el LED.	SendText "1"/"0", código del ESP32 y GPIO.

1.6 Evidencia a presentar

- ☐ 1 foto del circuito armado donde se identifique ESP32, LED y resistencia.



- ☐ 1 captura de pantalla de los bloques completos de App Inventor.



- ☐ Demostración en vivo: conectar por Bluetooth y ejecutar ON/OFF.
Ya está

- ☐ Si el docente no puede observar en vivo: video corto (15–30 s) mostrando conexión y cambio del LED.

<https://drive.google.com/file/d/1Skq8eQi-u-d26GvvYnh1ICNyH-ZMqDnb/view?usp=sharing>

Ejercicio 2 — Control de dos LEDs por Wi-Fi usando HTTP GET

Objetivo. Crear un servidor web en el ESP32 y controlar dos salidas digitales mediante cuatro rutas HTTP GET.

2.1 Hardware

Elemento	Conexión
LED 1	GPIO 4 → resistencia 220–330 Ω → LED → GND.
LED 2	GPIO 5 → resistencia 220–330 Ω → LED → GND.
ESP32	USB 5 V.
Red	ESP32, laptop y celular en la misma Wi-Fi.

2.2 Arduino IDE y servidor web

- Pegue el código del Ejercicio 2 del documento de códigos.
- Reemplace TU_SSID y TU_PASS por las credenciales de la red de trabajo.
- Compile y cargue el sketch.
- Abra el Monitor Serial a 115200.
- Anote la IP que aparece, por ejemplo 172.20.25.225. La IP puede cambiar después de reiniciar el ESP32.
- En un navegador de la misma red escriba `http://IP_DEL_ESP32/`.
- Pruebe los cuatro enlaces de la página: LED 1 ON, LED 1 OFF, LED 2 ON y LED 2 OFF.
- No continúe con App Inventor si los enlaces del navegador no controlan correctamente los LEDs.

2.3 MIT App Inventor — Designer

- Cree un proyecto nuevo: Ejercicio2_HTTP_GET.
- En Layout arrastre dos HorizontalArrangement. Configure Width = Fill parent y AlignHorizontal = Center.
- Dentro del primer HorizontalArrangement agregue dos Button: btnLed1On y btnLed1Off. Text: “LED 1 ON” y “LED 1 OFF”.
- Dentro del segundo HorizontalArrangement agregue btnLed2On y btnLed2Off. Text: “LED 2 ON” y “LED 2 OFF”.
- En Connectivity arrastre Web. Aparecerá como componente no visible: Web1.

2.4 MIT App Inventor — Blocks

- Para btnLed1On.Click: set Web1.Url to “`http://IP/led1/on`” y luego call Web1.Get.
- Para btnLed1Off.Click: set Web1.Url to “`http://IP/led1/off`” y luego call Web1.Get.
- Para btnLed2On.Click: set Web1.Url to “`http://IP/led2/on`” y luego call Web1.Get.
- Para btnLed2Off.Click: set Web1.Url to “`http://IP/led2/off`” y luego call Web1.Get.
- Use la IP real observada en el Monitor Serial; no copie una IP de otro equipo.

Idea clave: GET solicita una URL. Cada ruta del servidor representa una acción. Antes de culpar a la app, pruebe la misma URL en el navegador.

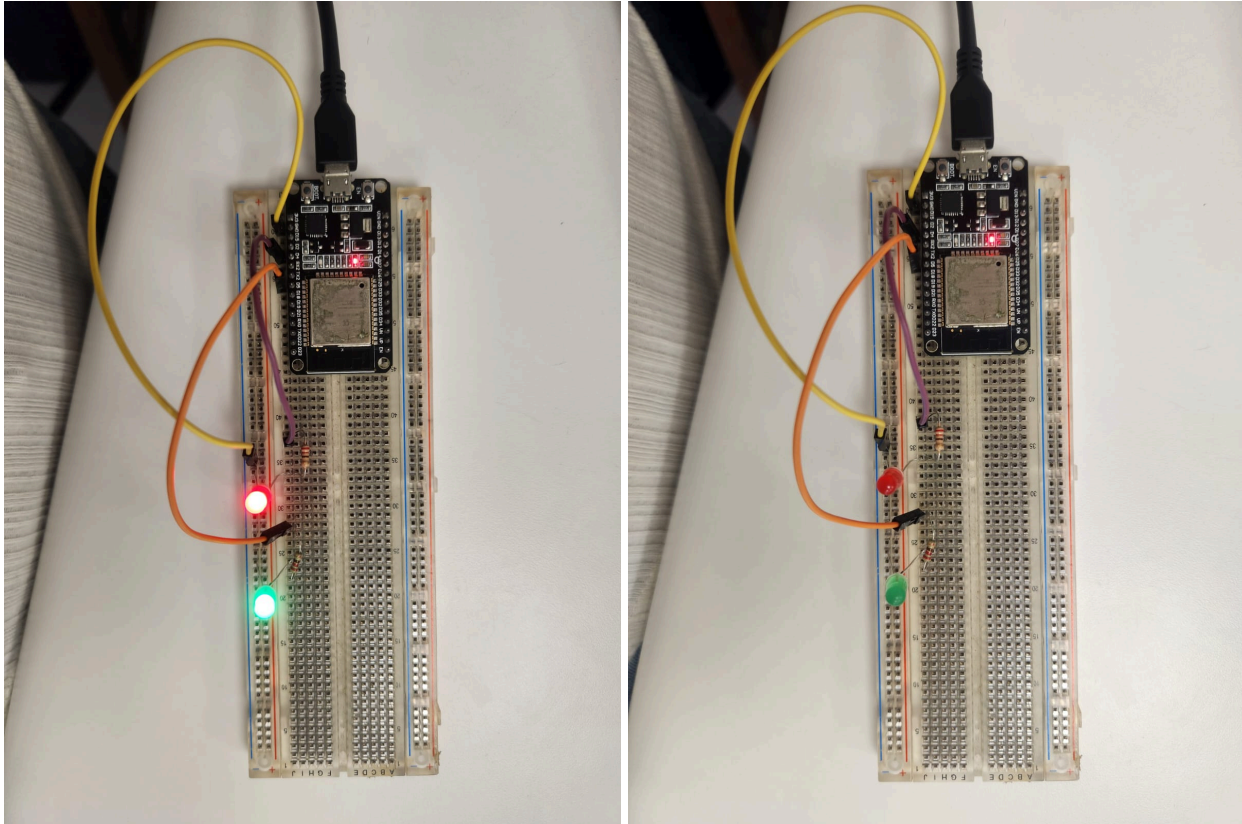
2.5 Diagnóstico paso a paso

Orden	Qué comprobar	Interpretación
1	Los LEDs funcionan con un sketch local.	Valida hardware.
2	El Monitor Serial muestra una IP.	Valida conexión Wi-Fi del ESP32.
3	<code>http://IP/</code> abre en navegador.	Valida el servidor web.

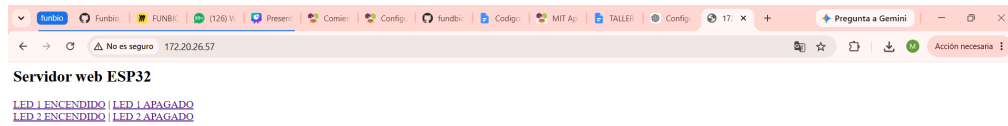
Orden	Qué comprobar	Interpretación
4	/led1/on, /led1/off, /led2/on, /led2/off funcionan.	Valida rutas y GPIO.
5	La app usa la misma IP y Web1.Get.	Valida App Inventor.
6	Si navegador funciona pero app no.	El problema está en URL/bloques/red del celular, no en LEDs.

2.6 Evidencia a presentar

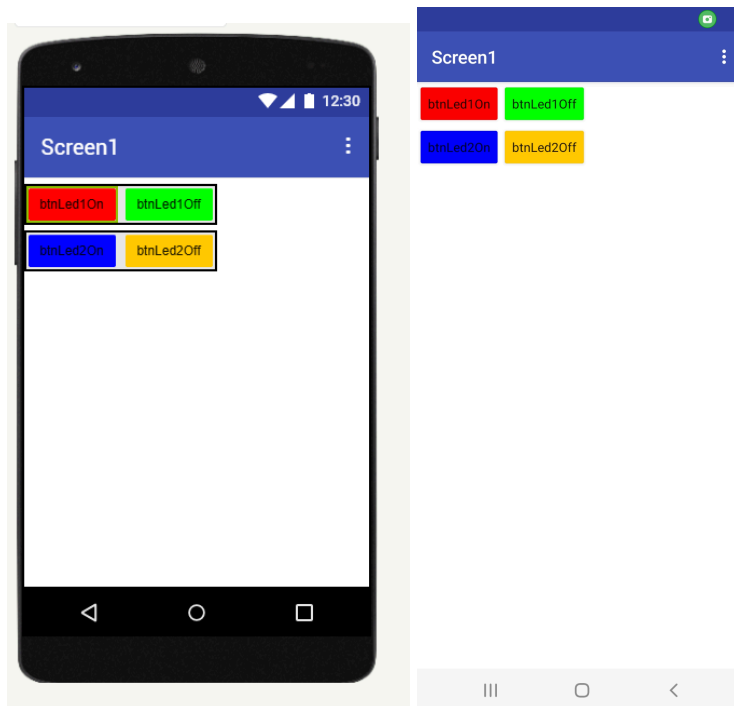
- ☐ Foto del circuito con los dos LEDs.



- ☐ Captura del navegador mostrando la página del ESP32.



- ☐ Captura de los cuatro bloques Click en App Inventor.



- ☐ Demostración en vivo: controlar ambos LEDs desde el celular.

<https://drive.google.com/drive/u/1/folders/1yWCNkq3V2CPyF1VtnkUa2w3yMGiCgLRc>

Ejercicio 3 — Control de un servomotor por Wi-Fi usando HTTP POST + JSON

Objetivo. Enviar desde un Slider un valor de 0 a 180, empaquetarlo como JSON, recibirlo en el ESP32 y mover un servo.

3.1 Hardware

Cable del servo	Conexión de taller
Rojo	+5 V
Marrón / negro	GND
Amarillo / naranja	GPIO 18 (señal)

Alimentación: No alimente el servo desde 3.3 V. Si el servo vibra, se reinicia el ESP32 o no logra moverse bajo carga, use una fuente externa de 5 V con GND común.

3.2 Validación obligatoria del servo antes de Wi-Fi

37. Cargue primero el código de prueba de servo del documento de códigos.
38. Verifique que el servo se mueva a tres posiciones: 0°, 90° y 180°.
39. Si esta prueba falla, no continúe con App Inventor. Revise alimentación, GND, GPIO 18 y cable de señal.
40. Si funciona, cargue el código completo del Ejercicio 3.

3.3 Preparación del firmware

41. Instale desde Library Manager: ArduinoJson y ESP32Servo.
42. En el código completo coloque TU_SSID y TU_PASS.
43. Compile, cargue y abra el Monitor Serial a 115200.
44. Anote la IP del ESP32.
45. El servidor debe exponer la ruta /servo y aceptar HTTP POST.

3.4 MIT App Inventor — Designer

46. Cree un proyecto nuevo: Ejercicio3_Servo_JSON.
47. Agregue Slider1. Configure MinValue = 0, MaxValue = 180 y ThumbPosition = 90.
48. Agregue Label1. Texto inicial sugerido: “Ángulo: 90°”.
49. En Connectivity agregue Web1.
50. En Sensors agregue Clock1. Configure TimerInterval = 300 y TimerEnabled = false.

3.5 MIT App Inventor — Blocks

51. En Variables cree initialize global Objetivo to 90. Use un bloque numérico de Math, no texto.
52. En Slider1.PositionChanged, actualice Label1.Text con join “Ángulo: ” + thumbPosition.
53. En el mismo evento: set global Objetivo to thumbPosition.
54. En el mismo evento: set Clock1.TimerEnabled to true.
55. Cree when Clock1.Timer.
56. Primera acción: set Clock1.TimerEnabled to false.
57. Segunda acción: set Web1.Url to “http://IP_DEL_ESP32/servo”.
58. Tercera acción: set Web1.RequestHeaders to una lista de listas: [[“Content-Type”, “application/json”]].
59. Cuarta acción: call Web1.PostText. El texto debe formar exactamente {“angulo”:VALOR}. Puede usar un join de tres entradas o dos join anidados.
60. Cree when Web1.GotText y coloque set Label1.Text to responseContent.

3.6 Por qué hay una lista dentro de otra lista

RequestHeaders espera una lista de encabezados; cada encabezado es a su vez una pareja nombre–valor. Por eso se representa como:

```
[ [ "Content-Type", "application/json" ] ]
```

La lista interna contiene el encabezado; la lista externa permite tener uno o varios encabezados.

3.7 Qué hace cada bloque importante

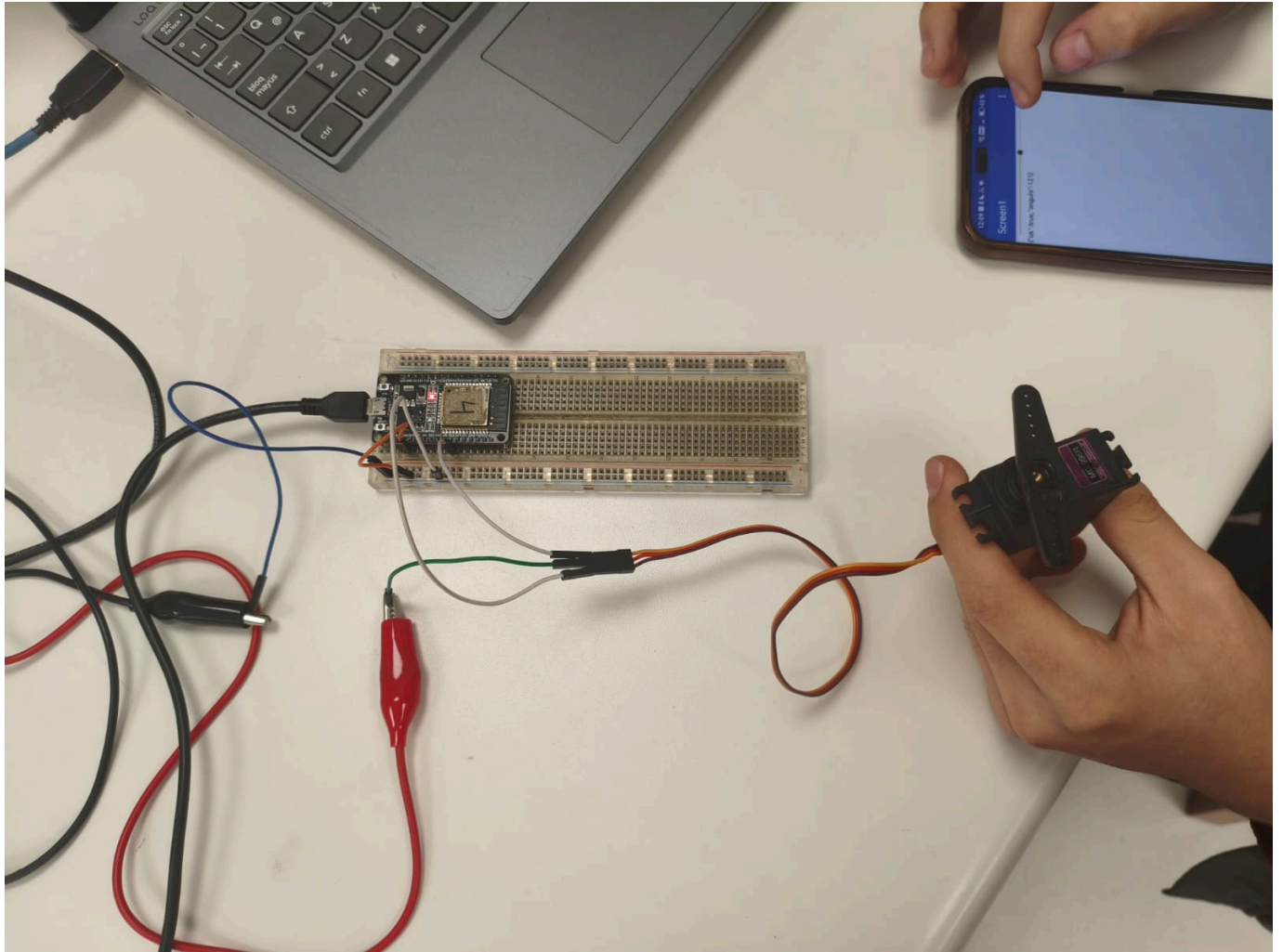
Bloque	Función
global Objetivo	Guarda el último ángulo seleccionado.
Clock1	Introduce una espera de 300 ms y evita enviar solicitudes continuamente mientras se arrastra el slider.
Web1.Url	Indica el endpoint /servo del ESP32.
RequestHeaders	Indica que el cuerpo enviado es JSON.
PostText	Envía el JSON mediante HTTP POST.
GotText	Se ejecuta cuando llega la respuesta del ESP32.
responseContent	Contiene la respuesta, por ejemplo {"ok":true,"angulo":88}.

3.8 Diagnóstico paso a paso

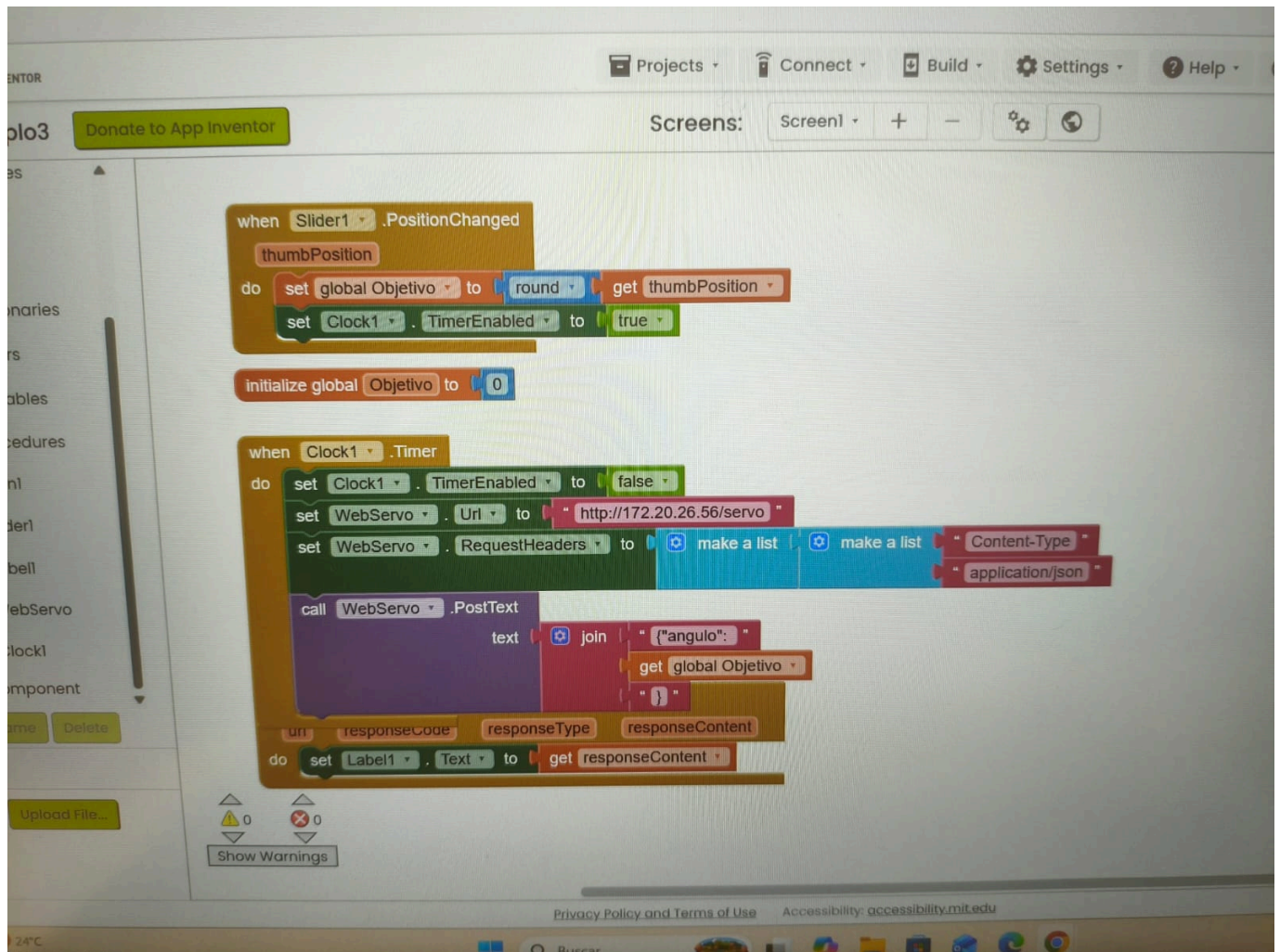
Orden	Observación	Qué concluye / qué revisar
1	Servo se mueve 0–90–180 con código mínimo.	Hardware/servo/GPIO correctos.
2	ESP32 obtiene IP.	Wi-Fi correcto.
3	Al mover slider, Monitor Serial no muestra petición.	Revisar IP, misma red, Clock y Web1.PostText.
4	Serial muestra "Texto recibido:" vacío.	Falta o está mal RequestHeaders Content-Type: application/json.
5	Serial muestra {"angulo":99} y "Ángulo interpretado: 99".	App → HTTP → JSON funciona.
6	Label muestra {"ok":true,"angulo":99}.	La respuesta ESP32 → app también funciona.
7	JSON llega bien pero servo no se mueve.	Volver a revisar alimentación o señal; comparar con prueba mínima.

3.9 Evidencia a presentar

- ☐ Foto del servo conectado al ESP32.



- ☐ Captura de todos los bloques de App Inventor.



- ☐ Captura del Monitor Serial mostrando un JSON recibido y el ángulo interpretado.
- ☐ Demostración en vivo o video corto moviendo el Slider y observando el servo.

https://drive.google.com/file/d/118oBJnKS7swKnf61A1gHnbwQOuwiahDh/view?usp=drive_link

Ejercicio 4 — Adquisición analógica en tiempo real: potenciómetro → flex sensor

Estructura. Este ejercicio tiene dos etapas: 4A es una práctica guiada con potenciómetro para validar ADC + HTTP GET; 4B es la adaptación asíncrona calificable con flex sensor, calibración y visualización.

4A — Práctica guiada con potenciómetro

4A.1 Conexión

Pin del potenciómetro	Conexión
Extremo 1	3.3 V
Pin central (cursor)	GPIO 34
Extremo 2	GND

4A.2 Validar primero el ADC

61. Cargue el código “4A-1 Prueba ADC”.
62. Abra el Monitor Serial a 115200.
63. Gire lentamente el potenciómetro.
64. Los valores deben cambiar y pueden aproximarse al rango 0–4095.
65. Si el valor queda fijo, revise el pin central, 3.3 V, GND y GPIO 34.

4A.3 Publicar el ADC en /readADC

66. Cargue el código “4A-2 Servidor ADC por Wi-Fi”.
67. Configure TU_SSID y TU_PASS.
68. Anote la IP del ESP32 en el Monitor Serial.
69. Desde un navegador abra `http://IP_DEL_ESP32/readADC`.
70. Gire el potenciómetro y actualice la página. El número debe cambiar.
71. No continúe con App Inventor hasta que /readADC funcione desde el navegador.

4A.4 MIT App Inventor — Designer

72. Cree un proyecto: Ejercicio4_ADC.
73. Agregue Label1. Texto inicial: “ADC: ---”.
74. En Connectivity agregue Web1.
75. En Sensors agregue Clock1.
76. Configure Clock1.TimerInterval = 500 y TimerEnabled = true.

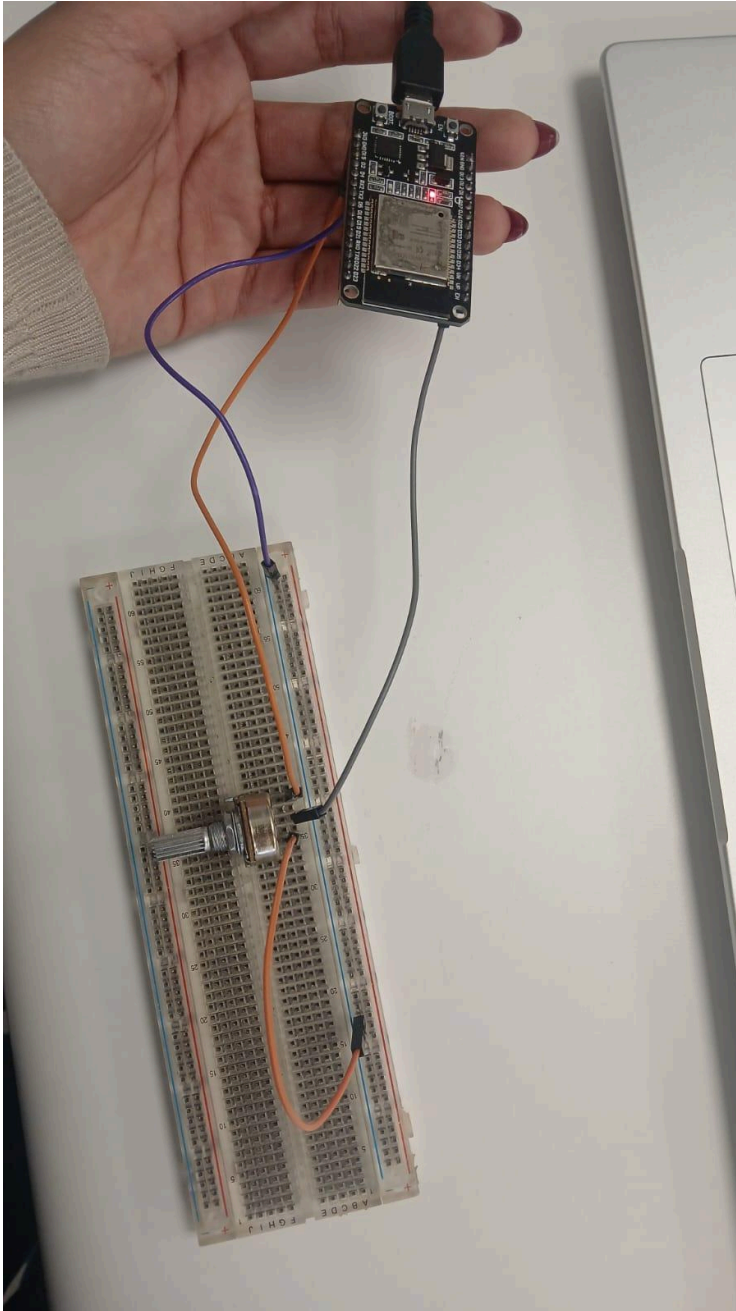
4A.5 MIT App Inventor — Blocks

77. Cree when Clock1.Timer.
78. Dentro: set Web1.Url to “http://IP_DEL_ESP32/readADC”.
79. Debajo: call Web1.Get.
80. Cree when Web1.GotText.
81. Dentro: set Label1.Text to responseContent. Opcional: use join “ADC: ” + responseContent.

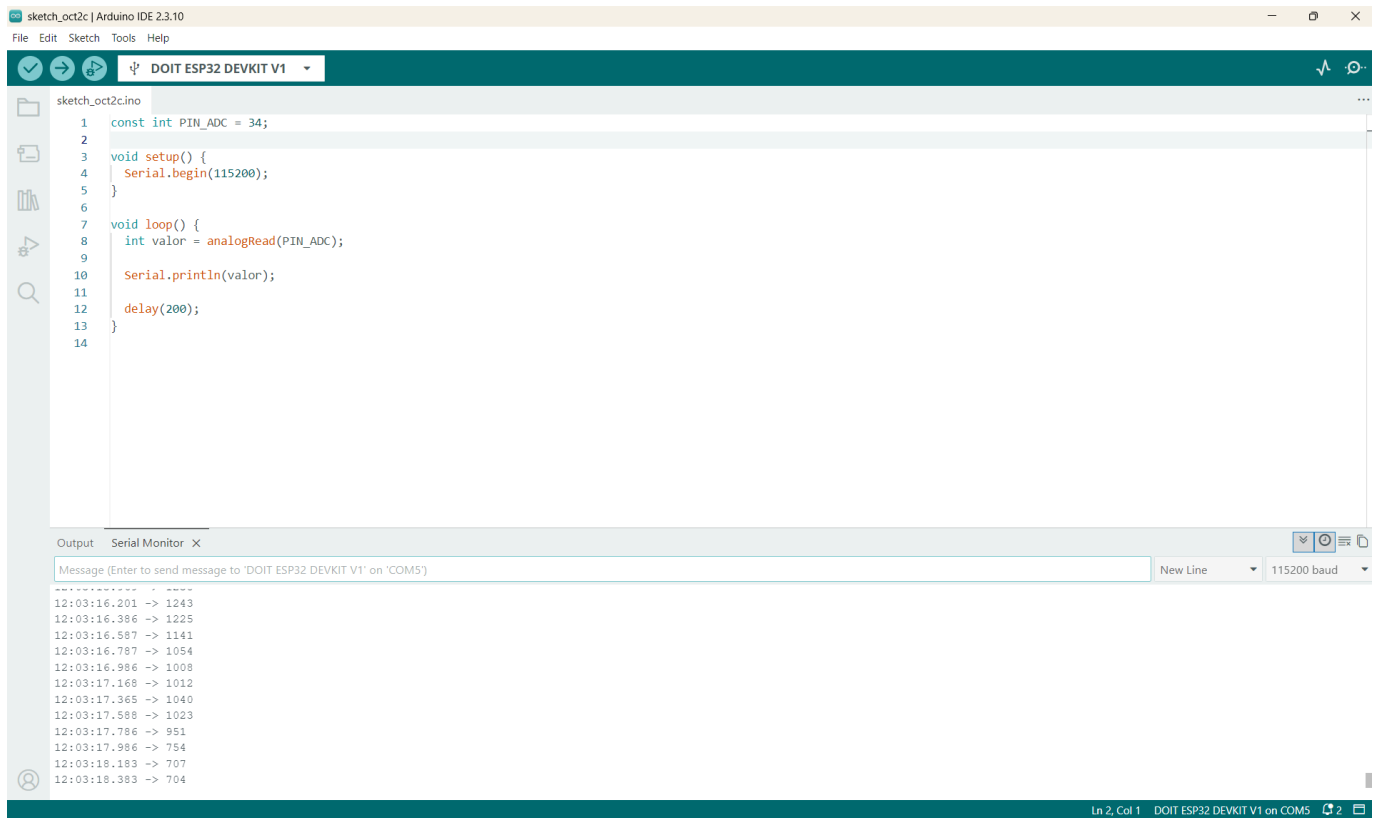
Flujo: Cada 500 ms el Clock ejecuta un GET. El ESP32 lee `analogRead(GPIO34)`, responde con texto y GotText actualiza el Label.

4A.6 Evidencia a presentar

- ☐ Foto del potenciómetro conectado.



- ☐ Captura del Monitor Serial mostrando variación del ADC.



The screenshot shows the Arduino IDE interface. The top menu bar includes 'File', 'Edit', 'Sketch', 'Tools', and 'Help'. The toolbar contains icons for checking, compiling, uploading, and serial monitor. The sketch is named 'DOIT ESP32 DEVKIT V1'. The code in the editor is as follows:

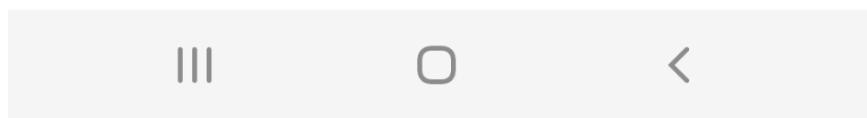
```
1  const int PIN_ADC = 34;
2
3  void setup() {
4    Serial.begin(115200);
5  }
6
7  void loop() {
8    int valor = analogRead(PIN_ADC);
9
10   Serial.println(valor);
11
12   delay(200);
13 }
14
```

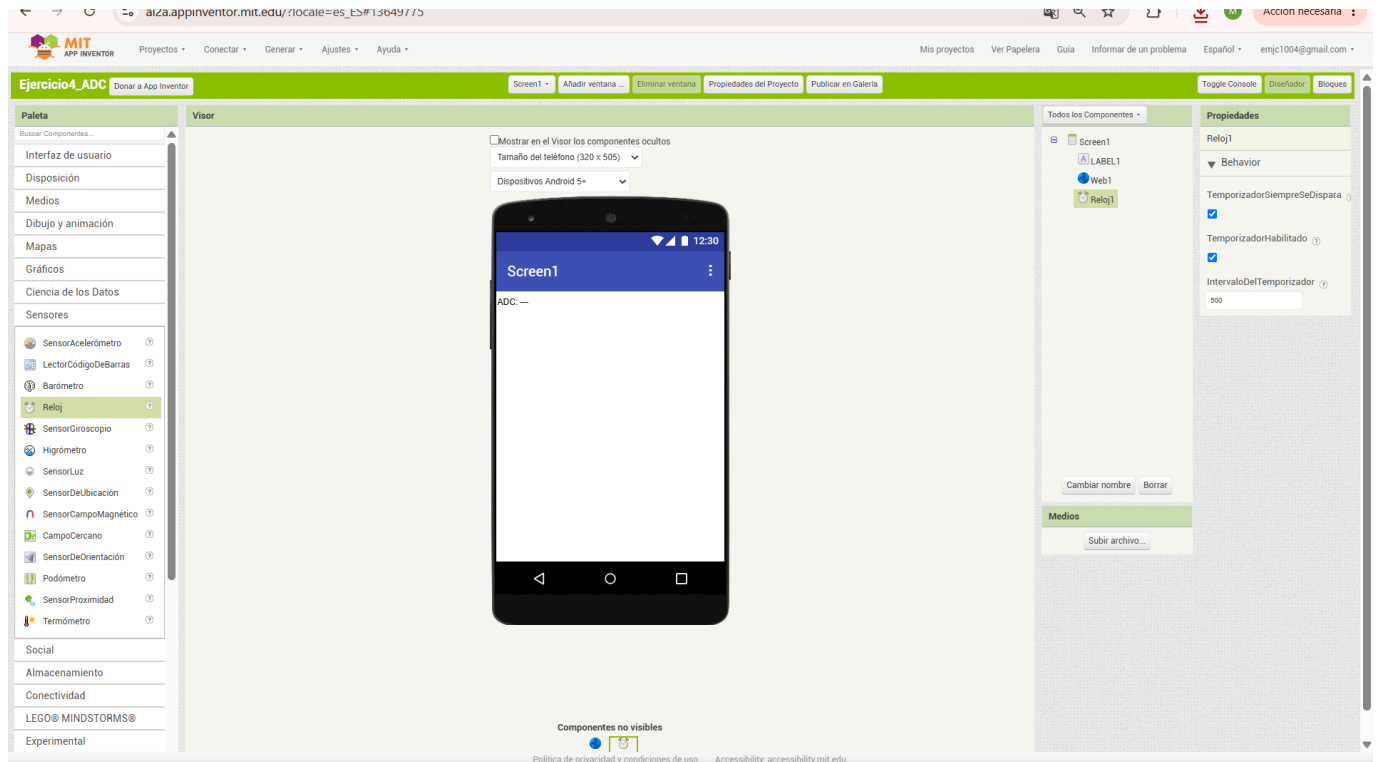
The 'Serial Monitor' window is open, showing a list of values being printed to the serial port at 115200 baud. The values are: 1243, 1225, 1141, 1054, 1008, 1012, 1040, 1023, 951, 754, 707, and 704. The status bar at the bottom indicates 'Ln 2, Col 1 DOIT ESP32 DEVKIT V1 on COM5'.

- ☐ Captura del navegador en /readADC



4095





- ☐ Captura de los bloques Clock1.Timer y Web1.GotText.

```

cuando Reloj1 . Temporizador
ejecutar
  poner Web1 . Url como "http://172.20.26.57/readADC"
  llamar Web1 . Obtener
  
```

```

cuando Web1 . ObtuvoTexto
  url códigoDeRespuesta tipoDeRespuesta contenidoDeRespuesta
ejecutar
  poner LABEL1 . Texto como tomar contenidoDeRespuesta
  
```

- ☐ Demostración breve: girar el potenciómetro y observar el valor en la app.

https://drive.google.com/file/d/1QKW-YiaBstli7uslJoltq2IW_4ZswO0_/view?usp=sharing

4B — Proyecto asíncrono calificable: monitor de flexión

Objetivo. Sustituir el potenciómetro por un flex sensor, calibrar su respuesta y visualizar flexión/ángulo en tiempo real. Esta etapa se alinea con el proyecto asíncrono del repositorio.

4B.1 Qué cambia respecto al potenciómetro

Aspecto	Potenciómetro (4A)	Flex sensor (4B)
Hardware	El potenciómetro ya es un divisor de tensión.	Necesita flex sensor + resistencia fija de 47 kΩ.
Variable física	Posición de perilla.	Flexión / curvatura / ángulo estimado.
ADC	GPIO 34 en la práctica guiada.	La guía original usa A0/GPIO36; puede usarse un ADC1 explícito.
Procesamiento	Valor crudo 0–4095.	Crudo → mV → calibración → ángulo.
App mínima	Label con Web.Get.	Además se solicita gauge y gráfica en vivo.

4B.2 Conexión del flex sensor

```

3.3 V
|
Flex sensor
|
+-----> ADC (GPIO36 en la guía original)
|
47 kΩ
|
GND

```

Nota: La orientación del divisor puede hacer que el valor aumente o disminuya al doblar. Lo importante es medir y calibrar.

4B.3 Procedimiento autónomo

82. Monte el divisor de tensión con el flex sensor y la resistencia de 47 kΩ.
83. Antes de Wi-Fi, lea el ADC en Monitor Serial con el sensor recto y doblado. Registre ambos valores.
84. Si dispone de multímetro, registre también el voltaje del punto medio en ambas posiciones.
85. Cargue el firmware asíncrono corregido del documento de códigos.
86. Configure la red con WiFiManager: si no existen credenciales guardadas, el ESP32 crea la red FlexAP con contraseña 12345678; conéctese y seleccione la Wi-Fi de trabajo.
87. Abra /readADC desde un navegador y confirme que el valor cambia al flexionar.
88. Calibre: determine mv_min y mv_max (o raw_min/raw_max) a partir de sus mediciones reales. No use valores de otro equipo sin medir.
89. Mapee el rango a 0–90° (o al rango que defina el equipo). Si la relación queda invertida, intercambie los límites del map.
90. En App Inventor, use Clock con intervalo cercano a 200 ms si trabaja con Web.Get.
91. Actualice Label con el valor recibido.

92. Agregue una gráfica histórica usando Chart + ChartData2D: cree un contador de muestra y en cada GotText agregue un punto (contador, valor).
93. Implemente un indicador de ángulo (gauge) mediante Canvas o la solución gráfica acordada para el taller.
94. Compruebe que la app responda de forma continua sin perder la conexión.

Correcciones/adaptaciones respecto al README del repositorio: La versión de códigos del taller corrige “text/plane” a “text/plain”; usa un GPIO ADC explícito para evitar ambigüedad; agrega mensajes de Monitor Serial e impresión de IP; y mantiene la calibración como parámetros que cada equipo debe medir. Estas son adaptaciones de robustez, no un cambio del objetivo del ejercicio.

4B.4 Diagnóstico paso a paso

Orden	Problema observado	Qué revisar primero
1	ADC fijo en 0 o 4095.	Cableado del divisor, 3.3 V/GND, nodo medio, pin ADC.
2	ADC cambia pero /readADC no abre.	Wi-Fi, IP, servidor, misma red.
3	Navegador funciona pero app no.	URL, Clock, Web.Get, GotText.
4	Valor cambia pero ángulo es absurdo.	Calibración min/max y orientación del map.
5	Gráfica no agrega puntos.	ChartData2D.AddEntry y contador de muestra.
6	La señal tiene mucho ruido.	Promedio de varias muestras, conexión firme, intervalo de muestreo.

4B.5 Entregables del proyecto asíncrono

- ☐ Video de demostración menor a 1 minuto mostrando flex sensor y app funcionando.
- ☐ Captura completa de los bloques de App Inventor.
- ☐ Código Arduino comentado (.ino).
- ☐ Tabla de calibración: condición/ángulo conocido, ADC, mV y valor mapeado.
- ☐ Captura de la app con valor numérico y gráfica (y gauge si corresponde).
- ☐ Informe breve en Markdown: conexión, calibración, rango obtenido, decisiones de diseño y principales errores encontrados.

Rúbrica sugerida para el proyecto asíncrono (20 pts)

Criterio	Puntos
Conexión y lectura estable del flex sensor	5
Servidor web funcional y accesible	4
App recibe datos en tiempo real (<300 ms)	4
Visualización clara (gauge + gráfico)	4
Documentación y video de demostración	3